Automated Recognition of Handwritten Str8ts Puzzles

Ryan El Aroud Chenwen Huang Klaas Bloem

Abstract

We present a fully automated Str8ts recognition framework combining deep segmentation and handwritten digit classification. Mask R-CNN isolates the puzzle grid, applies geometric normalization, and enables 81-cell patch extraction, followed by a CNN digit recognizer trained specifically on Str8ts handwriting. Across 25 unseen images, the system achieved an average per-grid accuracy above 90%, confirming robust performance under realistic recording conditions.

The proposed pipeline performs end-to-end Str8ts extraction and interpretation from unconstrained camera images. First, a fine-tuned Mask R-CNN detector localizes the grid and produces a pixel-level mask robust to varied lighting, background clutter, and non-orthogonal viewpoints. The corners of the grid are automatically inferred from the segmentation mask, and a homography transformation performs perspective rectification, mapping the captured grid to a normalized 256×256 canonical format.

Next, the rectified grid is subdivided into 81 uniformly sampled patches using a deterministic grid partitioning algorithm. Each patch is classified by a dedicated handwritten digit classifier (SmallDigitCNN), previously trained on puzzle-specific samples and augmented with transformations modeling blur, rotation. Beyond digits, the final model also distinguishes black cells enabling structural parsing rather than pure OCR.

1. Introduction

Str8ts is a logic puzzle resembling Sudoku but governed by consecutive numerical runs separated by blacked-out cells. From a computer vision perspective, Str8ts poses a significantly harder extraction challenge than printed Sudoku: its captured forms frequently involve handwritten digits of uneven stroke thickness, reflective paper surfaces, mixed ink colors, and perspective skew due to handheld cameras.

Automated puzzle recognition is essential for future mobile solvers, digital assistive tools, and large-scale dataset creation. Without reliable grid detection and digit parsing, downstream rule checking and automated solving cannot be performed. A robust vision pipeline must therefore handle cluttered backgrounds, table textures, shadow edges and inconsistent handwriting.

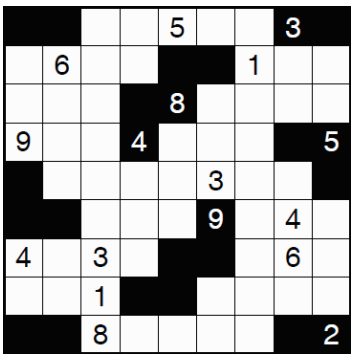

Figure 1. Initial Str8ts grid[1]

Our approach unifies deep segmentation with digit recognition into a single inference graph. Mask R-CNN provides strong localization and distortion correction, while a Str8ts-specific CNN digit classifier ensures accurate symbol retrieval. The pipeline consistently achieves a high recognition accuracy across varied test images.

2. Related Work

Most prior work on image-based puzzle recognition addresses Sudoku rather than Str8ts. Early pipelines use classical computer vision (edge or contour detection, perspective correction) for grid localization, followed by OCR or small CNNs for digit recognition [2, 3]. These systems work reasonably well on clean, printed puzzles, but struggle under real-world conditions such as uneven lighting, handwritten digits, or noisy backgrounds.

More recent works adopt deep learning at different stages, improving robustness. For instance, projects such as "Sudoku solver using computer vision and deep learning" (2024) combine image processing, neural-digit classification, and backtracking-based puzzle solving to handle natural images with varied lighting and distortion [4]. Similarly, a 2022 study proposes a method specifically designed to recognize Sudoku puzzles captured by smartphone cameras under different acquisition conditions [5].

A more advanced direction is hybrid perception-and-reasoning: e.g. NASR or similar frameworks integrate machine learning-based cell classification with a constraint

solver to perform joint inference from uncertain visual inputs; notably, the 2024 paper Perception-based constraint solving for Sudoku images uses a CNN to produce per-cell probabilities, then applies constraint programming to correct classifier mistakes and fill blank cells — demonstrating that combining perception with logic significantly improves reliability on noisy or handwritten Sudoku images [6].

In contrast, almost no work addresses vision-based recognition of Str8ts puzzles. While the combinatorial solving of Str8ts is treated in several solvers and helper apps (assuming a digital grid as input), the problem of interpreting raw images of Str8ts — including detection of black cells and handwritten digits — remains unaddressed. Our project fills this gap by adapting techniques from Sudoku-vision to Str8ts, but extending them with robust grid segmentation (via instance segmentation), richer cell-type classification, and logic-aware validation.

3. Data

The dataset consists of real photographs of Str8ts puzzle grids, intentionally captured under heterogeneous and imperfect conditions to reflect authentic user scenarios. Images were taken from multiple angles (frontal, oblique, tilted), with varying camera qualities, and under diverse lighting environments ranging from natural daylight to harsh indoor illumination and uneven exposure. These acquisition conditions were not controlled but rather embraced, as they represent the exact visual instability an automated recognition pipeline must overcome.

To further diversify handwriting and stroke variation, 15 different participants filled Str8ts grids manually, each using their own writing style, pen type, ink thickness, pressure, and spacing habits. This was a deliberate choice: unlike Sudoku datasets that often rely on uniform printed digits, Str8ts requires reliable recognition even when digits are irregular, lightly written, overlapping or partially erased. Consequently, digit variety, pen artifacts, and handwriting idiosyncrasies are central characteristics of this dataset rather than noise to be removed. All ground-truth cell labels were manually annotated to ensure precise supervision and full semantic consistency across the dataset.

Data augmentation was performed using Roboflow[7] including controlled variations in brightness, exposure, rotation, and mild perspective shift, reinforcing the model's ability to handle conditions typically found when users photograph puzzles on a desk, under a lamp, or with a handheld phone.

In summary, the dataset was constructed not as a clean laboratory corpus but as an intentionally noisy, human-produced visual collection. Its diversity directly supports the core objective of this project: achieving robust, real-world Str8ts grid understanding despite unstable lighting, irregular handwriting, and non-canonical capture angles.

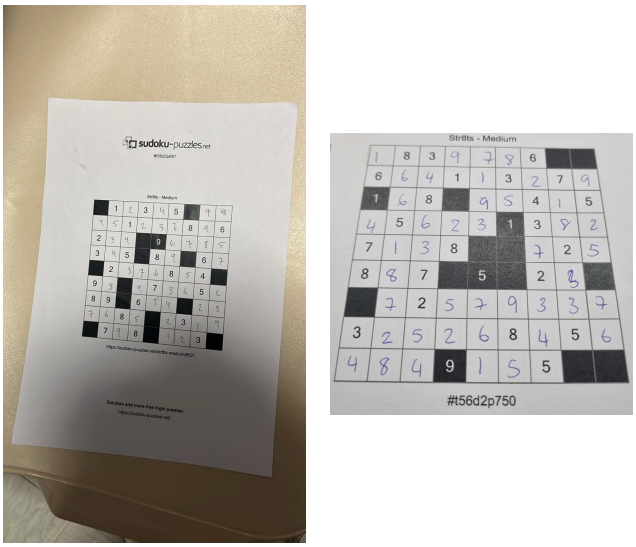

Figure 2. Comparison of two images picked randomly from the dataset

4. Methods

The overall pipeline is divided into two main modules: Grid Segmentation and Digit Classification. Both are implemented in PyTorch, with modular data loaders and a dynamic training configuration.

4.1. Corner Point Detection and Grid Segmentation

Our primary objective is to accurately locate the Str8ts board grid structure. Initial attempts employed traditional computer vision techniques by manual feature engineering. However, these methods proved inadequate for the specific geometry of Str8ts puzzles. Unlike standard chessboards, Str8ts puzzles contain solid black cells that disrupt the continuity of grid lines, causing linear detection algorithms to fail in those regions. Furthermore, environmental factors in real-world scenarios—such as variable lighting, cast shadows, and perspective distortion—rendered simple thresholding and geometric rules unstable. Consequently, we determined that a data-driven deep learning approach was necessary to learn a robust representation of the board's boundary features.

4.1.1. Mask Prediction

We selected Mask R-CNN as the foundational architecture for our corner detection system. This choice was made after evaluating and rejecting an alternative deep learning approach based on heatmap regression.

Comparison with Heatmap Regression: In our preliminary experiments, we attempted to model corner detection as a pixel-wise regression task, as shown in the figure 3 below. We converted ground truth corner coordinates into Gaussian heatmaps and trained a model to minimize the

difference between predicted and ground truth heatmaps. However, this method failed to converge effectively. The primary cause was the extreme class imbalance: the vast majority of the image is "background," which diluted the loss signal from the few "corner" pixels. Even after assigning higher loss weights to corner regions, the model struggled to focus on the sparse keypoints against the overwhelming background noise.

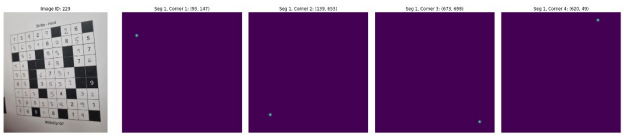

Figure 3. Predicted corner heatmaps for the four puzzle corners

Adoption of Instance Segmentation: To overcome the limitations of regression-based heatmaps, we pivoted to Mask R-CNN, an instance segmentation model. Instead of directly predicting sparse points, Mask R-CNN treats each corner as a distinct object instance. It predicts a binary mask that delineates the spatial extent of the corner. This region-based approach provides stronger supervision signals and effectively handles the background imbalance issue by focusing on Region of Interests (RoIs).

4.1.2. Corner Point Extraction

The output for mask R-CNN is a probability map, which requires post-processing to produce the precise four-vertex coordinates (x, y) necessary for grid reconstruction.

First, the predicted probability maps are thresholded at 0.5 to generate binary masks. Our initial approach was to extract the four corners by directly computing the minimum area bounding rectangle (e.g., using cv2.minAreaRect) that completely enclosed the foreground pixels. However, qualitative analysis revealed that this geometric approximation was insufficient. The method frequently produced "loose" bounding boxes—large squares that effectively enclosed the mask but failed to adhere to the irregular, perspective-distorted shapes of the chessboard corners. This resulted in significant drift between the detected vertices and the true corner locations.

Iterative IoU Optimization: To address this limitation, we implemented a refinement strategy based on iterative optimization. Instead of relying on a static geometric hull, we treat the four corner coordinates as adjustable parameters. The algorithm operates by iteratively perturbing the corner positions to maximize the Intersection over Union (IoU) between the polygon defined by the estimated corners and the predicted binary mask.

Mathematically, let P be the set of pixels in the predicted binary mask and $Q(\theta)$ be the set of pixels inside the quadrilateral defined by corner parameters θ. The objective is to find optimal parameters θ^* such that:

$$\theta^* = \underset{\theta}{\operatorname{argmax}} \frac{|P \cap Q(\theta)|}{|P \cup Q(\theta)|}$$

This optimization process effectively "shrink-wraps" the quadrilateral around the predicted mask, accommodating the non-rectilinear distortions caused by camera angles and ensuring a precise fit that aligns with the actual visual boundaries of the chessboard corner. The contrast between original corner points detection and refined corner points detection is shown in the figure 4 below.

Figure 4. Refined corner points detection

4.1.3. Grid Segmentation

Following the detection and precise extraction phases, the final step involves aggregating the individual corner instances to segment the full Str8ts chessboard grid into its constituent 9×9 cells. The set of the four extracted quadrilateral vertices, θ^*, provides a sparse but precise mapping of the board's topology.

A. Perspective Correction and Normalization

The quadrilateral boundary defined by θ^* precisely encloses the board, accounting for perspective distortion. The first step is to normalize this region. The detected quadrilateral area is cropped from the original image and transformed using a four-point perspective transformation (e.g., $cv2.warpPerspective$). This process maps the non-rectangular boundary into a perfectly square canonical view, thereby rectifying perspective skew and normalizing the appearance of the entire board.

B. Grid Partitioning

The Str8ts puzzle is a standard 81-cell (9×9) grid. Given the rectified square image, the system proceeds to partition it systematically. The most direct and geometrically accurate method is to divide each side of the square into nine equal segments, resulting in 81 equally sized, square-shaped cell patches.

4.2. Digit Recognition and Classification

After obtaining 81 patches from the image the patches are each normalized and converted to gray scale. In order to adjust for the noise at the edges of the pixels due to incorrect segmentation, patches are zoomed in to obtain the center 75% of the picture. This removes any outlining of the patches, which could influence the model's predictions. After obtaining the center region, the following features of each patch are extracted:

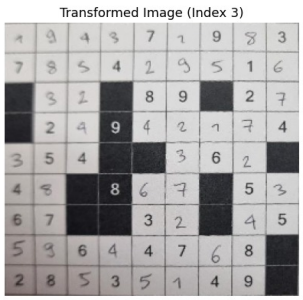

Figure 5. Transformed Str8ts board image to square

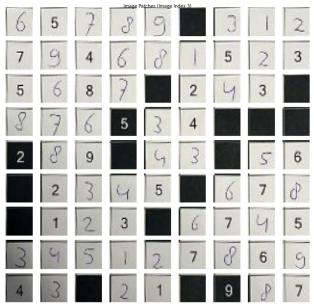

Figure 6. Cropping the Str8ts board into 81 patches for digit recognition

1) Dark pixel ratio: the percentage of pixels < 70 intensity.
2) Bright pixel ratio: the percentage of pixels > 200 intensity.
3) Standard deviation.
4) Pixel mean.

We trained a two-layer multi-layer perceptron (MLP) classifier on our four features to determine whether the patches are black, clue or digit patches. All patches that are labelled as a digit or clue cell are used as inputs to our digit recognition model. However, clue cells first need to be inverted in order for the model, which is pretrained on the MNIST dataset [7], to be used.

We first experimented using an EfficientNet-B0 architecture [4], pretrained on the MNIST dataset. After fine-tuning all network layers and on our custom dataset, and freezing different parts of the model, we found out that using a model of this size leads to overfitting and a slow to train model. This lead to us switching to a small CNN network, which improved both the accuracy and training time. Table 1 shows the CNN architecture. The outputs of the MLP and CNN result in a respresentation of 81 labels for each grid.

4.3. Full Pipeline

When two modules are combined together, the full pipeline is shown in figure 7 below.

5. Experiments

5.1. Corner Point Detection and Grid Segmentation

This section details the experimental setup, results as well as the relevant analyses for training the Mask R-CNN model, focusing on hyperparameter tuning to achieve optimal performance for corner mask segmentation. The primary goal was to identify the combination of optimizer and learning rate that resulted in the most rapid convergence and highest validation accuracy (measured by Intersection over Union, IoU).

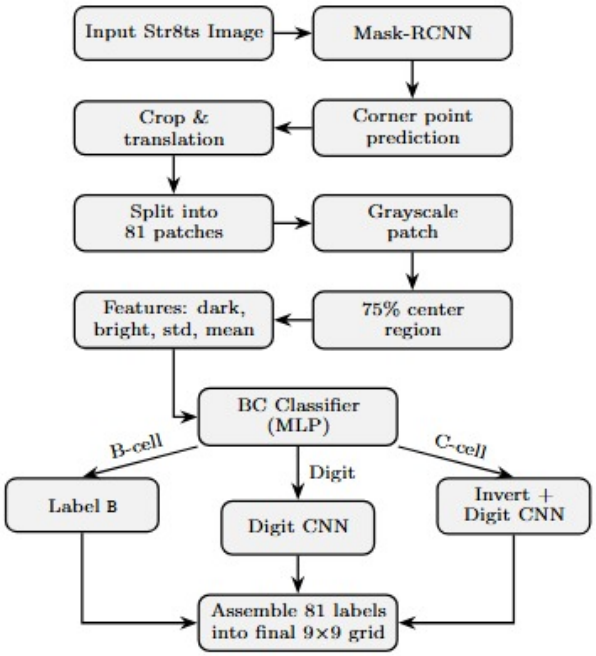

Figure 7. Full pipeline for Str8ts recognition

A. Parameter Setup

The model utilizes a ResNet50 backbone combined with a Feature Pyramid Network (FPN) , which extracts multi-scale feature maps and is critical for detecting corners that vary in apparent size due to camera perspective.

We used transfer learning by initializing the model with pre-trained COCO weights . The architecture was customized by replacing the standard prediction heads with `FastRCNNPredictor` and `MaskRCNNPredictor` layers, configured specifically for a two-class problem (foreground corner vs. background). The overall architecture is shown in the figure 8 below.

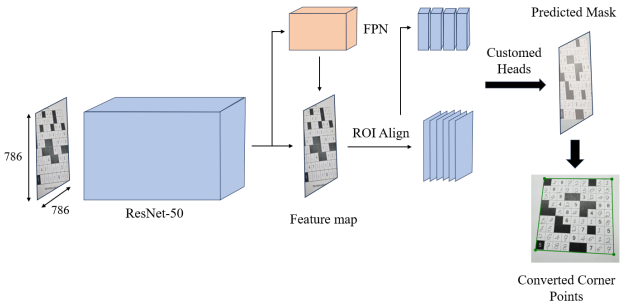

Figure 8. Mask R-CNN Model for mask prediction and corner points detection

The data are split into 80% for training, 10% for validation, and 10% for testing. The Mask R-CNN model utilized

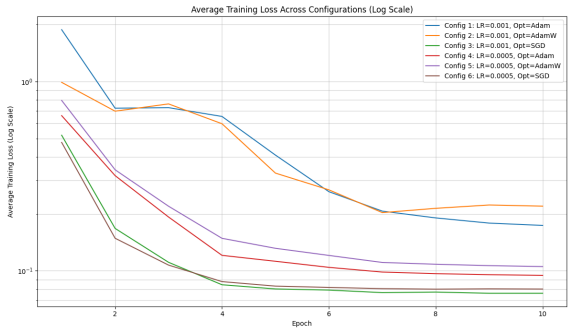

Figure 9. Training Loss over epochs

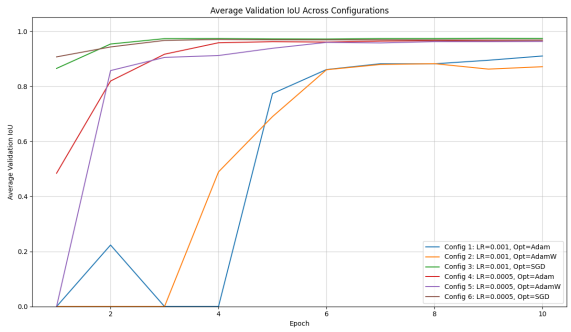

Figure 10. IOU over epochs

a ResNet50-FPN backbone and was initialized with pre-trained weights, with custom prediction heads configured for 2 classes.A crucial set of experiments involved systematically testing combinations of learning rates (LR) and optimizers across six configurations (Config 1–6) to determine their impact on model training stability and performance.

Table 1. Six Configuration for model training with epoch being 10

Config.	LR	Optimizer	Best Val. IoU
Config 1	0.001	Adam	0.9100
Config 2	0.001	AdamW	0.8821
Config 3	**0.001**	**SGD**	**0.9743**
Config 4	0.0005	Adam	0.9659
Config 5	0.0005	AdamW	0.9630
Config 6	0.0005	SGD	0.9701

B. Training Performance Analysis The Loss over epochs and the IOU over epoch plots are shown below in figure 9 and 10.

1). Average Training Loss

The analysis of the average training loss across the 10 epochs reveals significant differences in convergence speed and final minimum loss achieved: The two configurations using Stochastic Gradient Descent (SGD) (Config 3 and Config 6) demonstrated remarkably fast and stable convergence. Config 3 ($\text{LR} = 0.001, \text{Opt} = \text{SGD}$) achieved the lowest final loss value (≈ 0.0761), indicating the most efficient minimization of the combined loss function. The Adam and AdamW optimizers, particularly at the higher LR of 0.001 (Config 1 and 2), showed slower and more erratic loss reduction, often settling at higher minimum loss values. This suggests that while adaptive optimizers can be efficient, the simpler SGD with momentum (0.9 wave used in the model) was better suited for the specific structure and loss landscape of the fine-tuned Mask R-CNN architecture on this small, specialized dataset.

2). Validation IoU Performance

The primary evaluation metric was the Average Validation IoU, measuring the accuracy of the mask segmentation. The IoU graph clearly differentiates the performance of the configurations: Both SGD configurations (Config 3 and Config 6) rapidly achieved and maintained the highest IoU scores, plateauing above 0.96. Config 3 ($\text{LR} = 0.001, \text{Opt} = \text{SGD}$) achieved the overall best validation IoU of $\mathbf{0.9743}$ by the final epoch. The Adam/AdamW configurations at $\text{LR} = 0.001$ (Config 1 and 2) exhibited instability during early epochs, with several epochs registering IoU values of 0.0000. This indicates that the learning rate was potentially too aggressive for these adaptive optimizers, causing the bounding box/mask prediction to fail entirely early in training before recovering. The lower learning rate ($\text{LR} = 0.0005$) configurations (Config 4, 5, 6) showed more consistent performance than their higher LR counterparts, though only the SGD configuration (Config 6) approached the peak performance of Config 3.

Based on the combined analysis of training loss convergence and validation IoU accuracy, Config 3 ($\text{LR} = 0.001, \text{Optimizer} = \text{SGD}$) was selected as the optimal hyperparameter setting.

This configuration provided the fastest convergence to the lowest training loss while simultaneously achieving the highest validation IoU (0.9743), demonstrating the highest stability and accuracy in corner mask segmentation. This robust segmentation accuracy ($\text{IoU} > 0.97$) directly validates our methodological choice, confirming that the combination of Mask R-CNN and the subsequent iterative IoU optimization for corner point extraction provides a stable foundation for the downstream grid segmentation task.

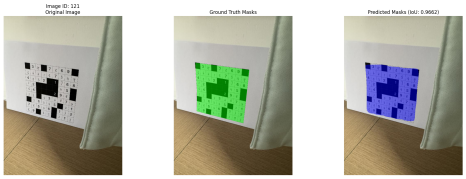

Figure 11. A case of model prediction

5.2. Digit Recognition and Classification

To determine whether a patch should be classified as a black, clue, or digit cell, we initially used the standard deviation and mean of the zoomed in patch. After seeing significant drops in grid prediction accuracy, we decided to use a designated MLP for this task. Using this MLP, the grid classification improved significantly for the majority of grids. However, there are outliers, especially images that are highly illuminated. These result in very bright grid pictures and degrade performance significantly, as shown in figure 13.

Pretraining the CNN and EfficientNet-B0 architecture on the MNIST dataset yielded an accuracy of respectively 99.2%, and 99.4% on the MNIST test data. Followed by finetuning on our own dataset got our CNN to an accuracy of 99.1% (figure 12) and the EfficientNet-B0 architecture to an accuracy of 95.1% on our data's validation set. Figure 14 illustrates an example of an annotated grid, including their corresponding correct/incorrect labels. Besides, the better accuracy obtained by the small CNN, the training time showed significant improvement.

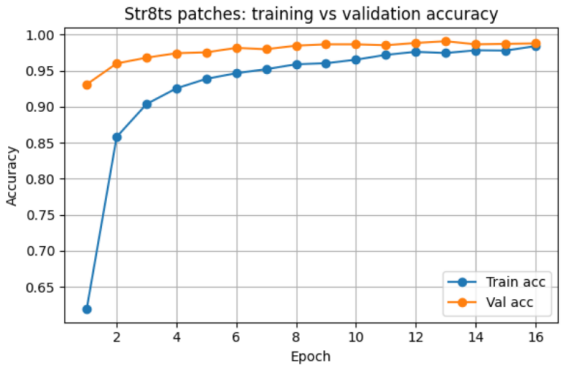

Figure 12. Digit Recognition accuracy plot

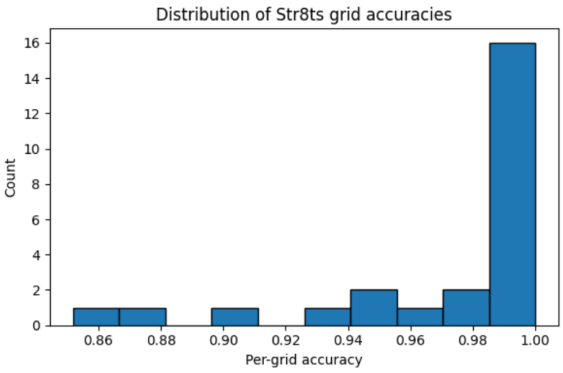

Figure 13. Intermediate results MLP + CNN classification

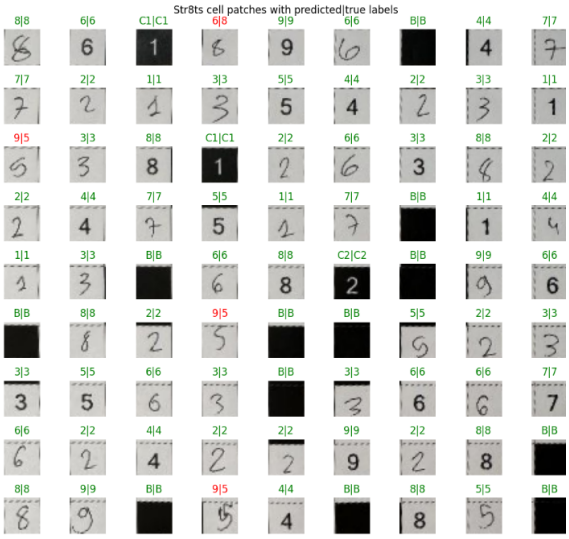

Figure 14. Grid example with corresponding labels

5.3. Full Pipeline Results

Combining our two pipelines for testing purposes leads to the results shown in figure 15. As mentioned before, when the pictures taken are highly illuminated, outliers arise. However, when a good picture is used, our pipeline shows very promising results.

The histogram shows a clear concentration of grids between 90% and 100% accuracy, indicating that the majority of puzzles are reconstructed almost perfectly. When grid segmentation is correct, the combination of Mask-RCNN, the BC classifier, and the digit CNN reliably identifies digits, black cells, and clue cells.

Conversely, the few low-accuracy cases are primarily due to segmentation failures: when the grid is poorly detected or rectified (imprecise corner localization, strong perspective distortion, clipped borders), the extracted patches no longer correspond accurately to their intended cells. Under these conditions, cell classification becomes extremely difficult, even for a well-trained recognition model, and overall performance drops significantly.

6. Conclusion

We developed a two-stage deep-learning pipeline for handwritten Str8ts recognition. The segmentation component successfully detects the puzzle region and corners, while the classification module accurately identifies path labels with 95.1% accuracy. Future work will focus on improving grid precision, expanding the dataset, and surpassing 90% full pipeline accuracy to enable full puzzle validation. The biggest hurdle remains grid annotation in low quality or highly illuminated pictures. An interesting implementation could be the addition of an algorithm checking the validity

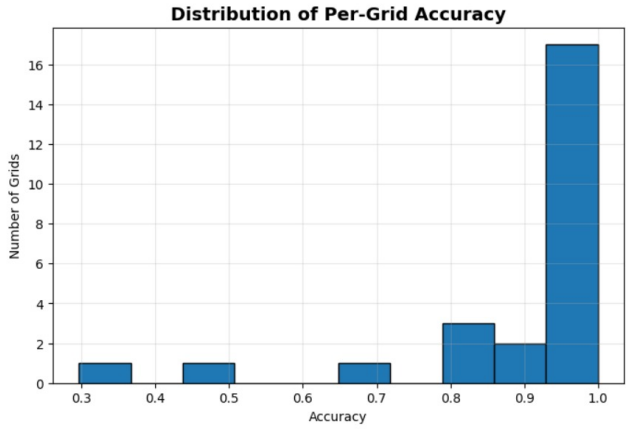

Figure 15. Results of the model applied to the test set made of 25 images.

of the puzzle, making it a useful tool for real life usage.

References

[1] *Str8ts*, Wikipedia, Accessed 2025.

[2] A. Rosebrock, *OpenCV Sudoku Solver and OCR*, PyImageSearch, 2020.

[3] A. Narayanaswamy, et al., "Sudoku Puzzle Recognition and Solving via Image Processing and Machine Learning," 2019.

[4] T. Jonas, et al., "Sudoku Solver Using Computer Vision and Deep Learning," JETIR, vol. 11, no. 2, 2024.

[5] J. Yu and H. Zhao, "Sudoku Recognition in Real-World Smartphone Capture Conditions," 2022.

[6] M. Mulamba, et al., "Perception-based Constraint Solving for Sudoku From Images," 2024.

[7] Roboflow, *Roboflow: Computer Vision Dataset Management, Annotation, and Augmentation Platform*, Available at: https://roboflow.com, Accessed 2025.